

Evaluación Parcial 2: Aplicación Full Stack Profesional

- **Asignatura:** PRO301 • Taller de Aplicaciones para Internet
- **Unidad:** 02 • Frontend Moderno, APIs y Legado
- **Fecha de Entrega:** Lunes 27 de abril de 2026
- **Duración:** 3 horas (Bloque de clase)
- **Modalidad:** Práctica Individual • Tema Libre
- **Docente:** Diego Obando

1. El Objetivo

Construir una aplicación web funcional que demuestre la integración completa de las tres capas fundamentales del desarrollo moderno: **Interfaz (Frontend)**, **Lógica de Servidor (API/Backend)** y **Persistencia (Base de Datos)**.

El tema del proyecto es **libre** (ejemplos: inventario de libros, registro de gastos, gestión de turnos, catálogo de productos, etc.), pero la estructura técnica debe cumplir rigurosamente con los requisitos detallados a continuación.

2. Requisitos Técnicos Obligatorios

Para que la aplicación sea evaluable, debe contener los siguientes tres componentes integrados:

A. Persistencia (Base de Datos)

- **Implementación:** Debe utilizar una base de datos real (PostgreSQL, MySQL, MongoDB, SQLite o similar). No se permiten listas en memoria RAM ni archivos JSON locales.
- **Estructura:** Al menos **una tabla o colección** con una Clave Primaria (PK) única y tipos de datos coherentes (ej: el precio debe ser numérico, la fecha tipo fecha).

B. Servidor de Recursos (API / Backend)

- **Arquitectura:** Debe exponer al menos **dos (2) endpoints** funcionales siguiendo el estándar REST:
 1. `POST /recurso` : Para recibir datos del frontend y guardarlos en la base de datos.
 2. `GET /recurso` : Para consultar la base de datos y devolver la lista de registros al frontend.
- **Formato:** La comunicación debe realizarse estrictamente en formato **JSON**.

C. Interfaz de Usuario (Frontend)

- **Semántica:** Uso de etiquetas HTML5 correctas (`<header>` , `<main>` , `<section>` , `<form>` , `<table>` o `<ul>` , `<footer>`).
- **Diseño:** CSS responsivo (Mobile-First) que se adapte a dispositivos móviles y escritorio.
- **Interactividad:**
 - Un **Formulario** para enviar datos al servidor.
 - Una **Visualización** (Lista, Tabla o Galería) que muestre los datos reales que vienen de la API.

3. Instrucciones Paso a Paso (Flujo de Ejecución)

Se recomienda seguir este orden para asegurar la estabilidad del proyecto:

1. **Definir el Negocio:** Elija un tema y decida qué datos va a guardar (ej: "Tarea: nombre, prioridad, fecha").
2. **Preparar la Base de Datos:** Cree la tabla en su motor de preferencia (ej: Supabase/PostgreSQL) con los campos necesarios.
3. **Construir el Backend:**
 - Configure el servidor.
 - Cree la ruta para recibir datos (`POST`).
 - Cree la ruta para enviar datos (`GET`).
 - Pruebe las rutas con una herramienta de inspección (ej: Swagger o extensión .http).
4. **Construir el Frontend:**
 - Maquete el formulario y la lista en HTML.
 - Aplique estilos CSS responsivos.
 - Use JavaScript para conectar el formulario con el `POST` y la lista con el `GET` .
5. **Validación Final:** Verifique que al recargar la página los datos sigan ahí (Persistencia) y que la consola del navegador no muestre errores.

4. Rúbrica de Evaluación

Criterio	%	Excelente (Logrado)	Insuficiente (No Logrado)
Modelado y Persistencia	25%	Tabla con PK y tipos correctos. Los datos sobreviven al reinicio del sistema.	Los datos se pierden al reiniciar o no usa una base de datos real.
Arquitectura de API	25%	Rutas REST semánticas (POST/GET), manejo de JSON y separación clara de lógica.	Rutas confusas, no usa verbos HTTP correctamente o el servidor no responde.
Frontend y UX	20%	HTML semántico, CSS responsivo y diseño profesional (consistencia visual).	HTML basado solo en divs, no es responsivo o la jerarquía visual es inexistente.
Integración Full-Stack	15%	El "puente" funciona: el Front consume la API y actualiza la vista dinámicamente.	El Front tiene datos "fijos" (hardcodeados) que no vienen de la base de datos.
Robustez y Validación	10%	El sistema valida que los datos no estén vacíos y usa Status Codes correctos (200, 400, 404).	El servidor "explota" (Error 500) ante datos inválidos o incompletos.
Orden y Git	5%	Repositorio GitHub con commits lógicos y estructura de archivos profesional.	Entrega desordenada, sin historial de Git o archivos sueltos.

5. Criterios de "Suma y Baja" de Puntos

Suma puntos (+)

- **Relaciones Técnicas:** Implementar una relación entre dos tablas (Llave Foránea).
- **Capa de Seguridad:** Validar que los datos de entrada no sean maliciosos o de tipo erróneo en el servidor.
- **Documentación:** Incluir en el README del proyecto una breve explicación de cómo ejecutarlo localmente.

Baja puntos (-)

- **Uso Crítico de IA:** Entregar código con comentarios generados por IA (ej: `# implementar aquí`) que no fueron completados por el estudiante.
- **Texto Meta:** Dejar textos en la interfaz que digan "Ejemplo", "Prueba", "Tarea de Diego" o similares. La aplicación debe parecer un producto real.
- **Entrega fuera de plazo:** El repositorio debe estar disponible al cierre del bloque de clase.

6. Método de Entrega

1. Subir el código a un repositorio en **GitHub**.
2. El repositorio debe ser **público** o compartido con el docente.
3. Enviar el enlace del repositorio a través del canal oficial de la asignatura.